



# CHAPTER– 05

## BASIC TRAVERSAL AND SEARCH TECHNIQUE

Prepared by-  
Safrun Nesa Saira  
Lecturer  
Dept. of CSE, DIIT

# CONTENTS

- Techniques for Binary trees, Techniques for Graphs

# TECHNIQUE FOR BINARY TREE

# BINARY TREE TRAVERSAL

- Techniques only applicable to binary trees
- Examining every node in the given data object instance
- This technique referred to as *traversal method*

# BINARY TREE TRAVERSAL METHOD

1. Inorder Traversal
  2. Preorder Traversal
  3. Postorder Traversal
- Inorder Traversal
    - Traverse left subtree
    - Visit the root
    - Traverse right subtree

# BINARY TREE TRAVERSAL METHODS

- Preorder Traversal
  - Visit the root
  - Traverse left subtree
  - Traverse right subtree
- Postorder Traversal
  - Traverse left subtree
  - Traverse right subtree
  - Visit the root

# INORDER TRAVERSAL: ALGORITHM

```
treenode = record
{
    Type data; // Type is the data type of data.
    treenode *lchild; treenode *rchild;
}

1  Algorithm InOrder(t)
2  // t is a binary tree. Each node of t has
3  // three fields: lchild, data, and rchild.
4  {
5      if t ≠ 0 then
6      {
7          InOrder(t → lchild);
8          Visit(t);
9          InOrder(t → rchild);
10     }
11 }
```

---

**Algorithm 6.1** Recursive formulation of inorder traversal

# PREORDER & POSTORDER TRAVERSAL: ALGORITHMS

```
1  Algorithm PreOrder(t)
2  // t is a binary tree. Each node of t has
3  // three fields: lchild, data, and rchild.
4  {
5      if t ≠ 0 then
6      {
7          Visit(t);
8          PreOrder(t → lchild);
9          PreOrder(t → rchild);
10     }
11 }
```

```
1  Algorithm PostOrder(t)
2  // t is a binary tree. Each node of t has
3  // three fields: lchild, data, and rchild.
4  {
5      if t ≠ 0 then
6      {
7          PostOrder(t → lchild);
8          PostOrder(t → rchild);
9          Visit(t);
10     }
11 }
```

# BINARY TREE TRAVERSAL

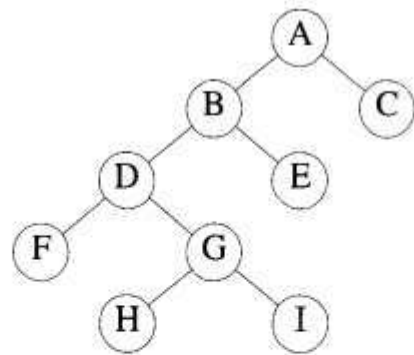


Figure 6.1 A binary tree

Inorder traversal  
= ?

Preorder traversal  
= ?

Postorder traversal  
= ?

# BINARY TREE TRAVERSAL

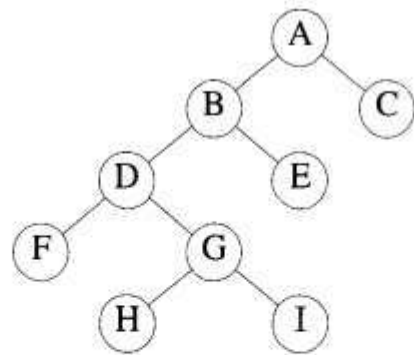


Figure 6.1 A binary tree

Inorder traversal  
= FDHGIBEAC

Preorder traversal  
= ?

Postorder traversal  
= ?

# BINARY TREE TRAVERSAL

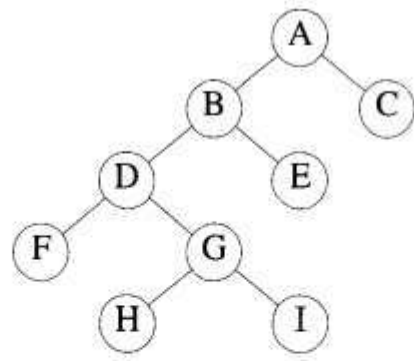


Figure 6.1 A binary tree

Inorder traversal  
= FDHGIBEAC

Preorder traversal  
= ABDFGHIEC

Postorder traversal  
= ?

# BINARY TREE TRAVERSAL

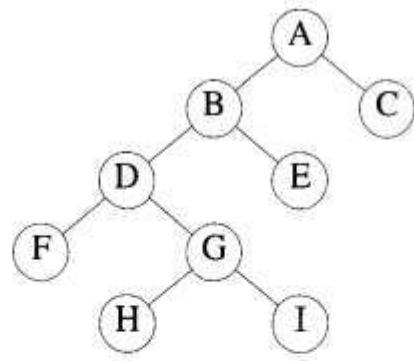


Figure 6.1 A binary tree

Inorder traversal  
= FDHGIBEAC

Preorder traversal  
= ABDFGHIEC

Postorder traversal  
= FHIGDEBCA

# TECHNIQUES FOR GRAPH

# TECHNIQUES FOR GRAPH

A fundamental problem concerning graphs is the reachability problem. In its simplest form it requires us to determine whether there exists a path in the given graph  $G = (V, E)$  such that this path starts at vertex  $v$  and ends at vertex  $u$ . A more general form is to determine for a given starting vertex  $v \in V$  all vertices  $u$  such that there is a path from  $v$  to  $u$ . This latter problem can be solved by starting at vertex  $v$  and systematically searching the graph  $G$  for vertices that can be reached from  $v$ . We describe two search methods for this.

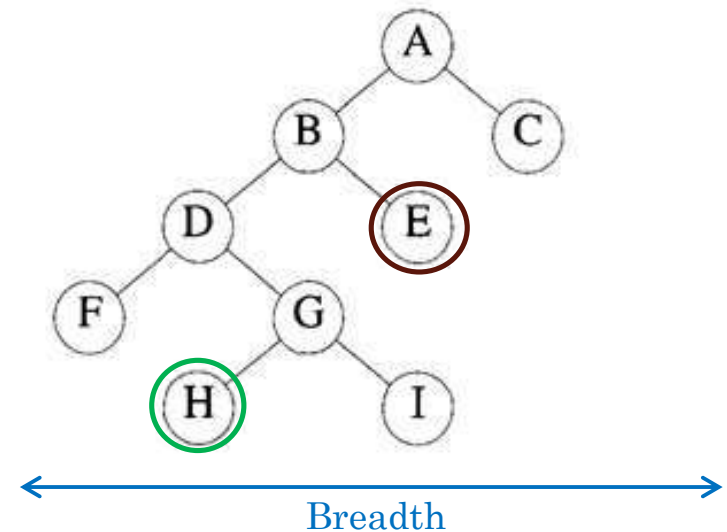
- 1 Breadth First Search

- 2 Depth First Search

# BREADTH FIRST SEARCH BFS

# 1. BREADTH FIRST SEARCH (BFS)

- BFS explores nodes nearest the root before exploring nodes further away
- For the given graph, nodes are explored in the order:  
A B C D E F G H I
- So, here node **E** is explored before node **H**



# BFS: RULES

- BFS follows the following rules-

1. Select an unvisited node  $s$ , visit it, make it the root node in a BFS tree. Now this level of this node is called the current level.
2. From each node  $x$  in the current level (in the order in which the same level nodes were visited previously), visit all the unvisited neighbors of  $x$ . Now the level of the newly visited nodes is the current level.
3. Repeat the previous steps until no more nodes can be visited.
4. If there are still unvisited nodes, repeat from step 1.

# BFS: FEATURES OR HOW TO DO BFS

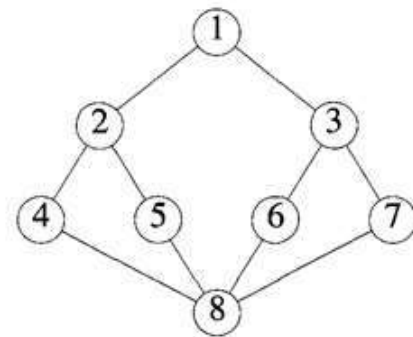
## ○ Features

```
Put the root node on a queue;  
while (queue is not empty){  
    Remove a node from the queue;  
    if (node is a goal node)  
        return success;  
    Put all children of node on the queue;  
}  
return failure;
```

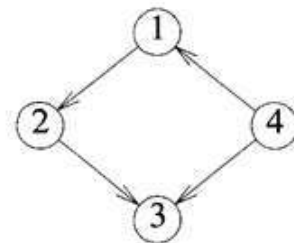
- Before starting to explore level  $n$ , the queue holds all the nodes at level  $n-1$
- In a typical tree, the number of nodes at each level increases exponentially with the depth
- Memory requirements may be infeasible
- When the method succeeds, it does not give the path

# BFS: AN EXAMPLE

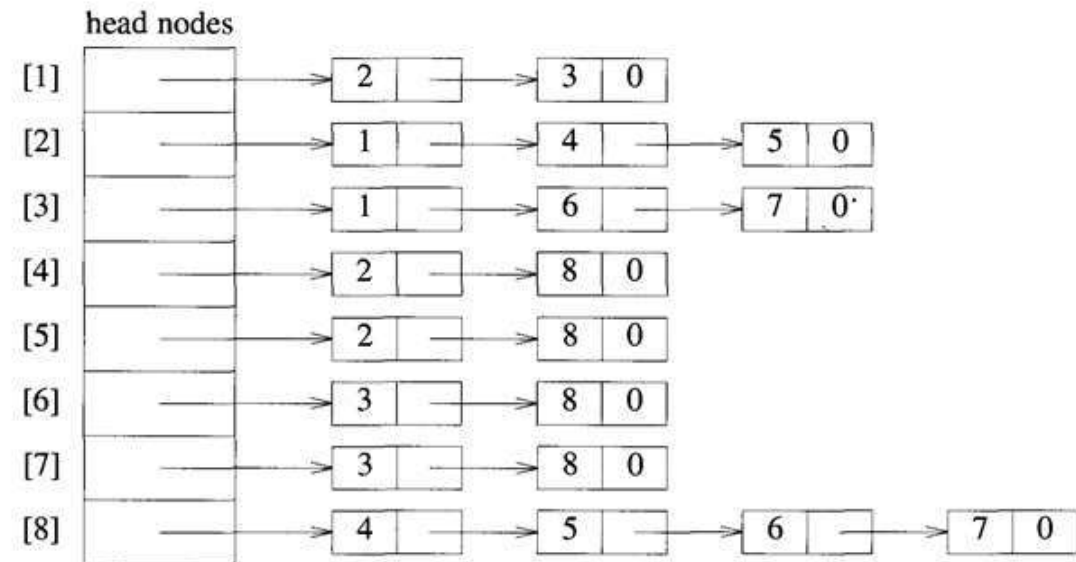
**Example 6.1** Let us try out the algorithm on the undirected graph of Figure 6.4(a). If the graph is represented by its adjacency lists as in Figure 6.4(c), then the vertices get visited in the order 1, 2, 3, 4, 5, 6, 7, 8. A breadth first search of the directed graph of Figure 6.4(b) starting at vertex 1 results in only the vertices 1, 2, and 3 being visited. Vertex 4 cannot be reached from 1.



(a) Undirected graph  $G$



(b) Directed graph



(c) Adjacency list for  $G$

Figure 6.4 Example graphs and adjacency lists

# BFS: ALGORITHM

```
1  Algorithm BFS( $v$ )
2  // A breadth first search of  $G$  is carried out beginning
3  // at vertex  $v$ . For any node  $i$ ,  $visited[i] = 1$  if  $i$  has
4  // already been visited. The graph  $G$  and array  $visited[ ]$ 
5  // are global;  $visited[ ]$  is initialized to zero.
6  {
7       $u := v$ ; //  $q$  is a queue of unexplored vertices.
8       $visited[v] := 1$ ;
9      repeat
10     {
11         for all vertices  $w$  adjacent from  $u$  do
12         {
13             if ( $visited[w] = 0$ ) then
14             {
15                 Add  $w$  to  $q$ ; //  $w$  is unexplored.
16                  $visited[w] := 1$ ;
17             }
18         }
19         if  $q$  is empty then return; // No unexplored vertex.
20         Delete  $u$  from  $q$ ; // Get first unexplored vertex.
21     } until(false);
22 }
```

## BFS: TIME COMPLEXITY

If adjacency lists are used, then all vertices adjacent from  $u$  can be determined in time  $d(u)$ , where  $d(u)$  is the degree of  $u$  if  $G$  is undirected and  $d(u)$  is the out-degree of  $u$  if  $G$  is directed. Hence, when vertex  $u$  is being explored, the time for the **for** loop of line 11 of Algorithm 6.5 is  $\Theta(d(u))$ . Since each vertex in  $G$  can be explored at most once, the total time for the **repeat** loop of line 9 is  $O(\sum d(u)) = O(e)$ . Then  $visited[i]$  has to be initialized to 0,  $1 \leq i \leq n$ . This takes  $O(n)$  time. The total time is therefore  $O(n + e)$ . If adjacency matrices are used, then it takes  $\Theta(n)$  time to determine all vertices adjacent from  $u$  and the time becomes  $O(n^2)$ . If  $G$  is a graph such that all vertices are reachable from  $v$ , then all vertices get explored and the time is at least  $O(n + e)$  and  $O(n^2)$  respectively. Hence,  $T(n, e) = \Theta(n + e)$  when adjacency lists are used, and  $T(n, e) = \Theta(n^2)$  when adjacency matrices are used.

# BFS: APPLICATION

1. Shortest Path and Minimum Spanning Tree for unweighted graph
2. Peer to Peer Networks
3. Crawlers in Search Engines
4. Social Networking Websites
5. GPS Navigation systems
6. Broadcasting in Network
7. Cycle detection in undirected graph
8. Path Finding

# BFS: THEOREM

**Theorem 6.2** Algorithm BFS visits all vertices reachable from  $v$ .

**Proof:** Let  $G = (V, E)$  be a graph (directed or undirected) and let  $v \in V$ . We prove the theorem by induction on the length of the shortest path from  $v$  to every reachable vertex  $w \in V$ . The length (i.e., number of edges) of the shortest path from  $v$  to a reachable vertex  $w$  is denoted by  $d(v, w)$ .

**Basis Step.** Clearly, all vertices  $w$  with  $d(v, w) \leq 1$  get visited.

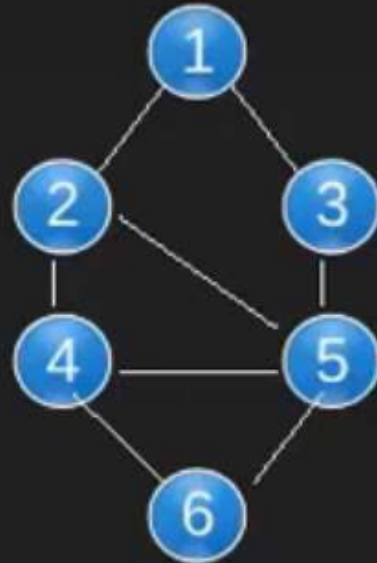
**Induction Hypothesis.** Assume that all vertices  $w$  with  $d(v, w) \leq r$  get visited.

**Induction Step.** We now show that all vertices  $w$  with  $d(v, w) = r + 1$  also get visited.

Let  $w$  be a vertex in  $V$  such that  $d(v, w) = r + 1$ . Let  $u$  be a vertex that immediately precedes  $w$  on a shortest  $v$  to  $w$  path. Then  $d(v, u) = r$  and so  $u$  gets visited by BFS. We can assume  $u \neq v$  and  $r \geq 1$ . Hence, immediately before  $u$  gets visited, it is placed on the queue  $q$  of unexplored vertices. The algorithm doesn't terminate until  $q$  becomes empty. Hence,  $u$  is removed from  $q$  at some time and all unvisited vertices adjacent from it get visited in the **for** loop of line 11 of Algorithm 6.5. Hence,  $w$  gets visited.  $\square$

# A BFS SIMULATION USING QUEUE

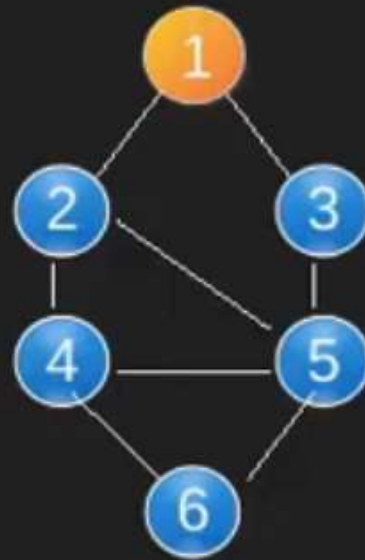
# BFS: SIMULATION USING QUEUE



Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |

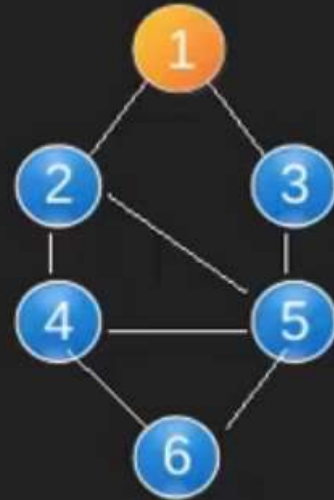
Queue :



Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 |

Queue : 1

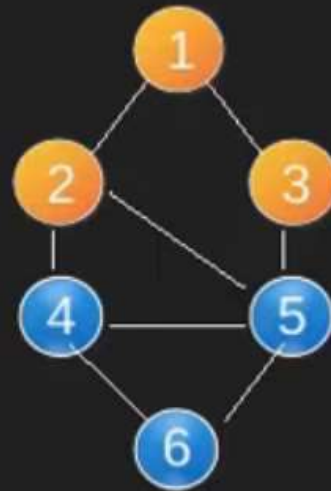


Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 |

Queue :

Print : 1



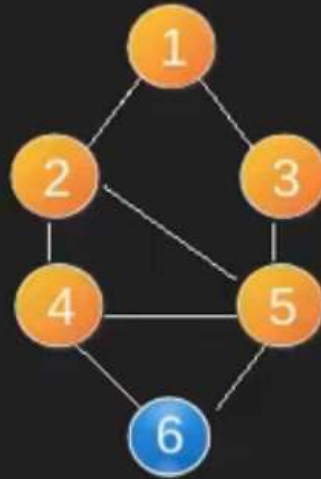
Visited :

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 | 0 |

Queue : 2 3

After removing 1 from queue and printing it, we enqueue its non-visited adjacentNodes

Print : 1

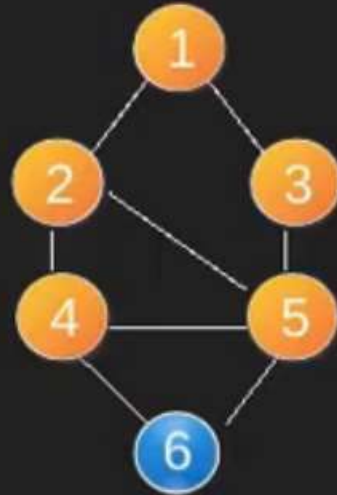


Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 0 |

Queue : 3 4 5

Print : 1 2

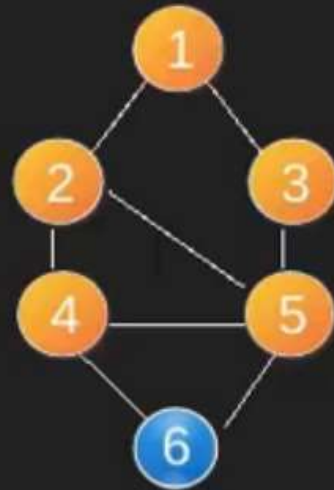


Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 0 |

Queue : 4 5

Print : 1 2 3

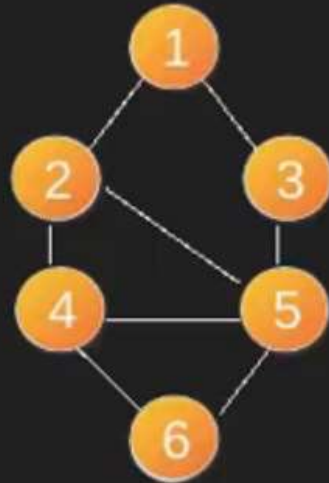


Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 0 |

Queue : 5

Print : 1 2 3 4

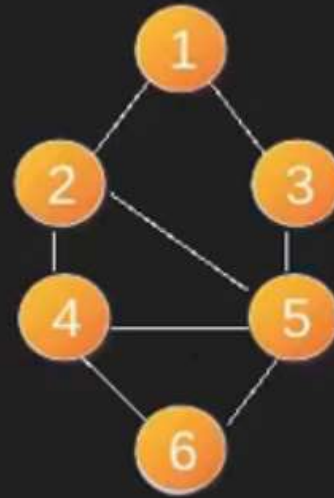


Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 |

Queue : 5 6

Print : 1 2 3 4

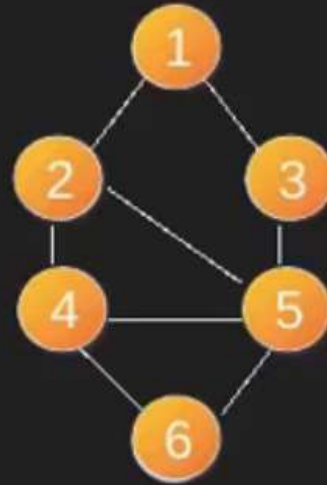


Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 |

Queue : 6

Print : 1 2 3 4 5



Visited : 

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 |

Queue :

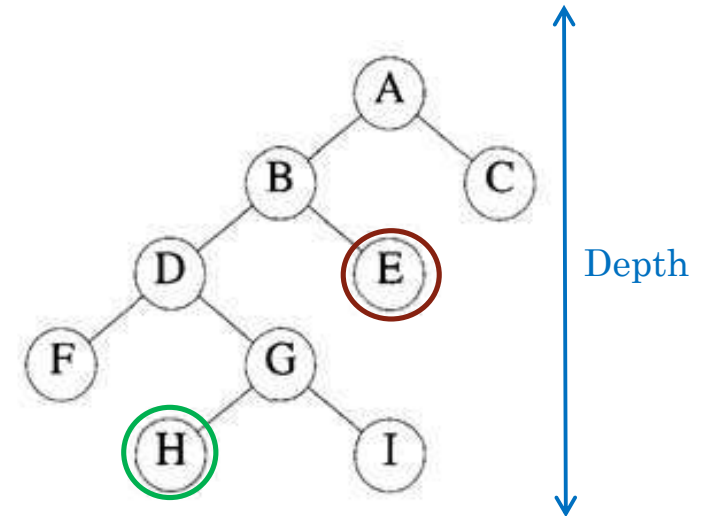
Print : 1 2 3 4 5 6

# DEPTH FIRST SEARCH

## DFS

## 2. DEPTH FIRST SEARCH (DFS)

- DFS explores a path all the way to a leaf node, before backtracking and exploring another path
- For the given graph, nodes are explored in the order:  
A B D F G H I E C
- So, here node **H** is explored before node **E**



# DFS: RULES

- DFS follows the following rules-
  1. Select an unvisited node  $s$ , visit it, make it the current node.
  2. Find an unvisited neighbor of the current node, visit it, and make it the new current node.
  3. If the current node has no unvisited neighbor, then backtrack to its parent, and make that the new current node. And then repeat the above two steps until no more nodes can be visited.
  4. If there are still unvisited nodes, repeat from step 1.

# DFS: FEATURES OR HOW TO DO DFS

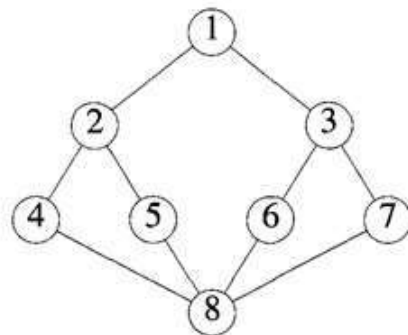
## ○ Features

```
Put the root node on a stack;  
while (stack is not empty){  
    Remove a node from the stack;  
    if (node is a goal node)  
        return success;  
    Put all children of node on the stack;  
}  
return failure;
```

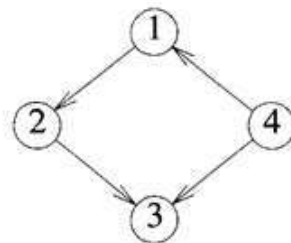
- At each step, the stack contains a path of nodes from the root node.
- The stack must be large enough to hold the longest path, that is, the maximum depth of search.

# DFS: AN EXAMPLE

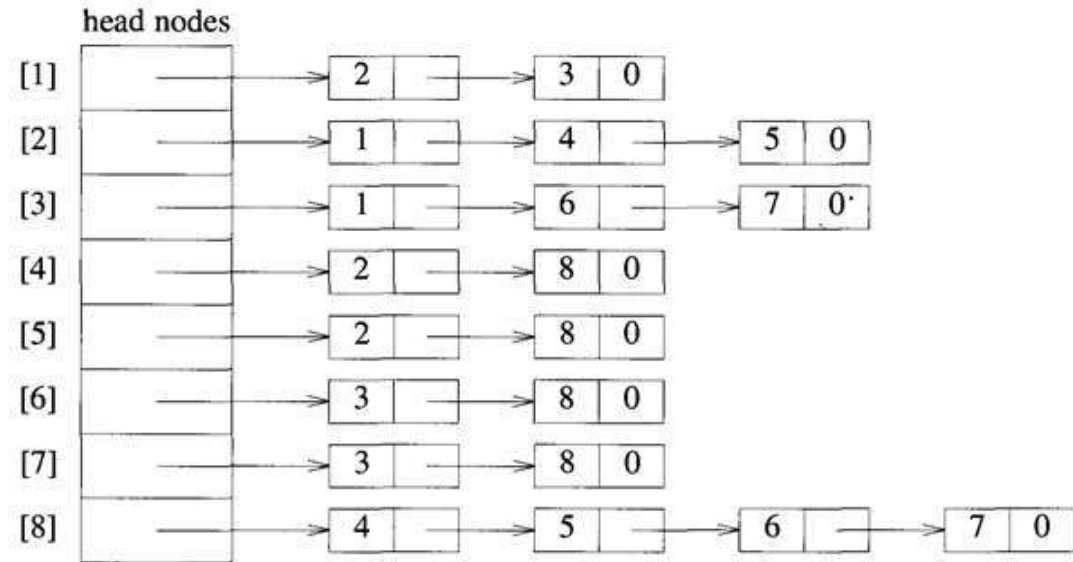
**Example 6.2** A depth first search of the graph of Figure 6.4(a) starting at vertex 1 and using the adjacency lists of Figure 6.4(c) results in the vertices being visited in the order 1, 2, 4, 8, 5, 6, 3, 7.



(a) Undirected graph  $G$



(b) Directed graph



(c) Adjacency list for  $G$

Figure 6.4 Example graphs and adjacency lists

# DFS: ALGORITHM

```
1  Algorithm DFS( $v$ )
2  // Given an undirected (directed) graph  $G = (V, E)$  with
3  //  $n$  vertices and an array  $visited[ ]$  initially set
4  // to zero, this algorithm visits all vertices
5  // reachable from  $v$ .  $G$  and  $visited[ ]$  are global.
6  {
7       $visited[v] := 1$ ;
8      for each vertex  $w$  adjacent from  $v$  do
9          {
10             if ( $visited[w] = 0$ ) then DFS( $w$ );
11          }
12 }
```

# DFS: TIME COMPLEXITY

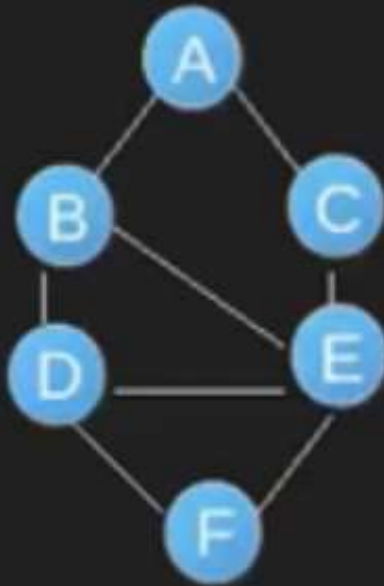
One can easily prove that DFS visits all vertices reachable from vertex  $v$ . If  $T(n, e)$  and  $S(n, e)$  represent the maximum time and maximum additional space taken by DFS for an  $n$ -vertex  $e$ -edge graph, then  $S(n, e) = \Theta(n)$  and  $T(n, e) = \Theta(n + e)$  if adjacency lists are used and  $T(n, e) = \Theta(n^2)$  if adjacency matrices are used

# DFS: APPLICATION

- Some applications of DFS are-
  1. Minimum spanning tree and all pair shortest path tree for weighted graph
  2. Detecting cycle in a graph
  3. Path Finding
  4. Topological Sorting
  5. Solving puzzles with only one solution, such as mazes

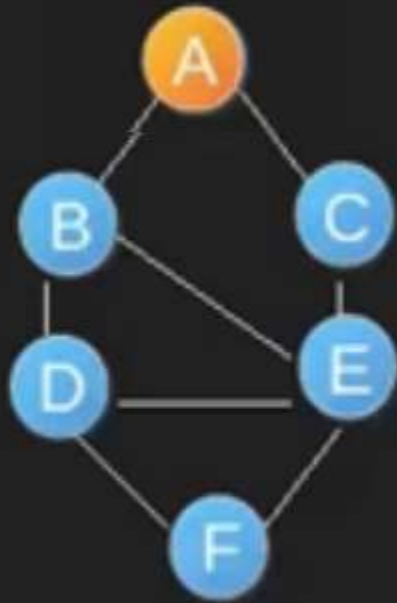
# A DFS SIMULATION USING STACK

# DFS: SIMULATION USING STACK



STACK

OUTPUT :



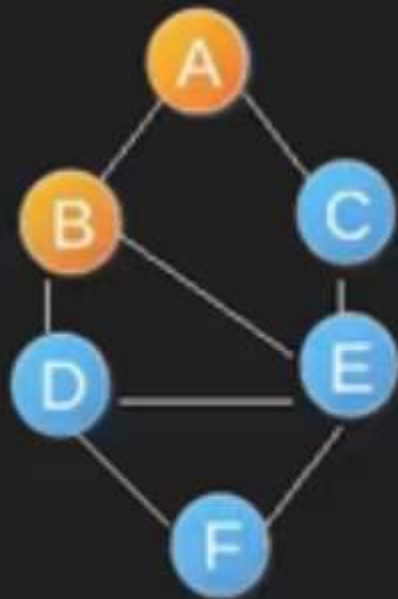
● : unvisited

● : visited



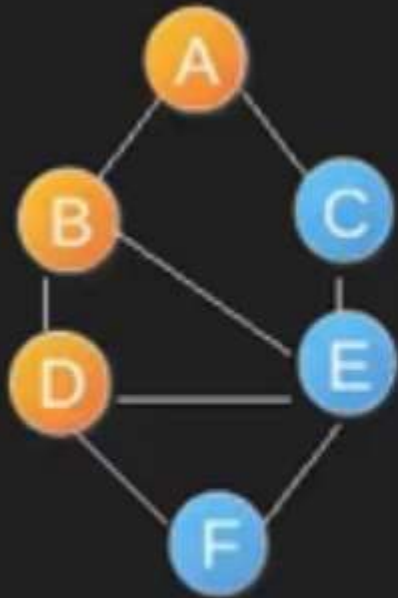
STACK

OUTPUT : A



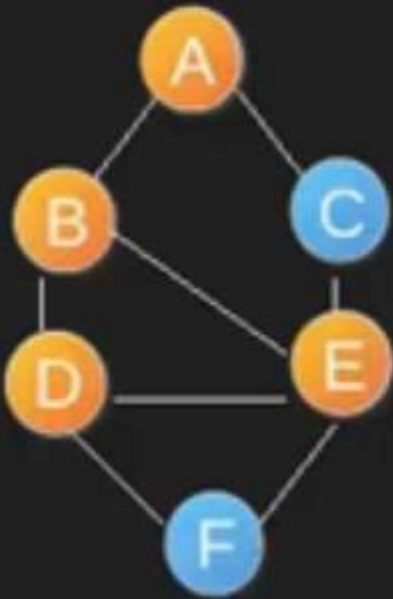
STACK

OUTPUT : A B



STACK

OUTPUT : A B D

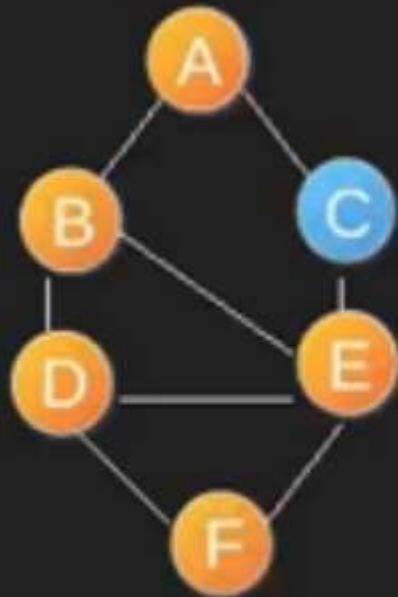


Note : E is a neighbour  
of D



STACK

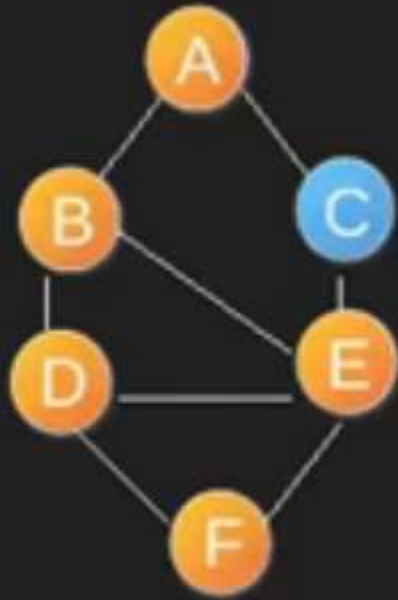
OUTPUT : A B D E



F  
E  
D  
B  
A

STACK

OUTPUT : A B D E F

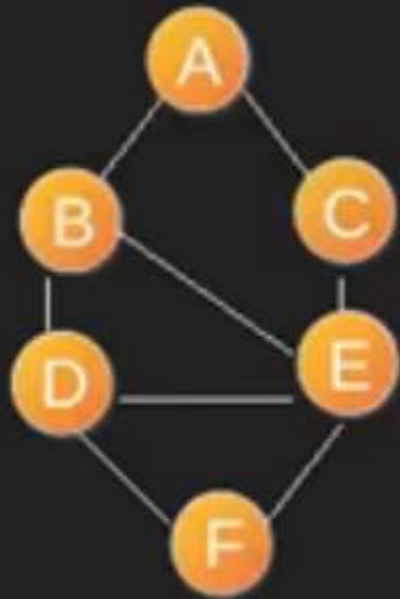


Note : F is removed from the stack



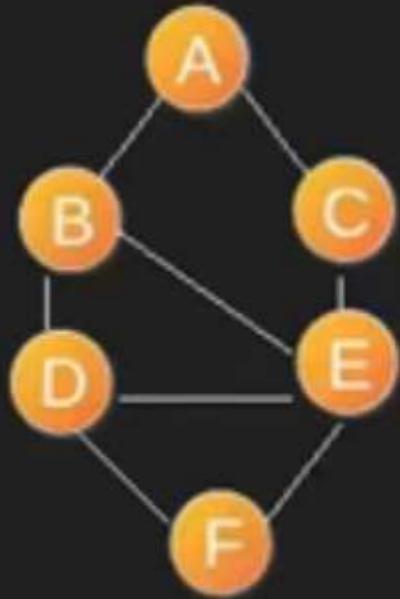
STACK

OUTPUT : A B D E F



STACK

OUTPUT : A B D E F C



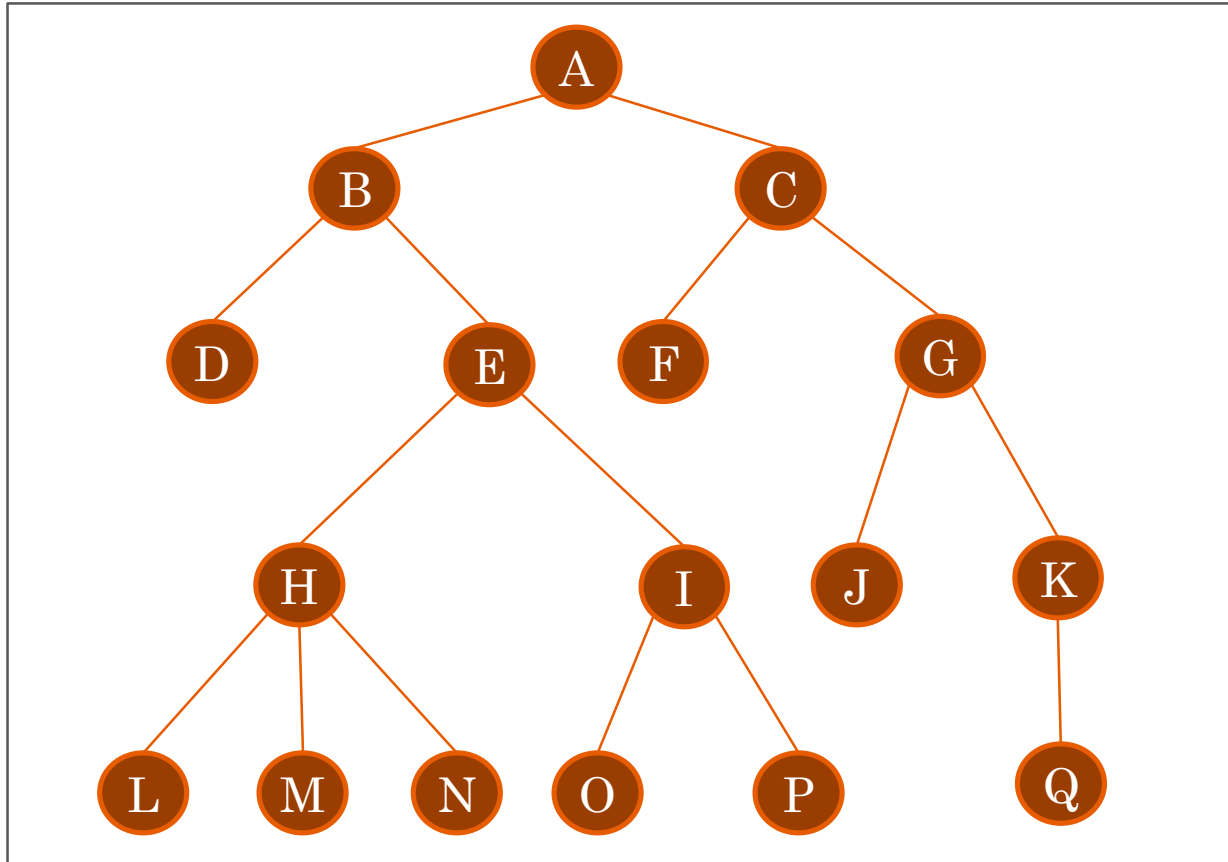
C, E, D, B and A are one by one removed from stack. Since all nodes are visited, no more nodes are added.



STACK

OUTPUT : A B D E F C

## EXERCISE

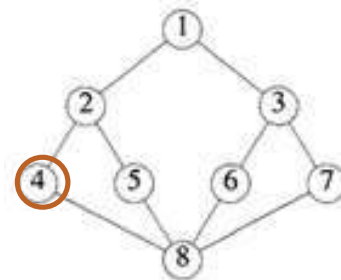


In BFS nodes are explored in the order  
= ?

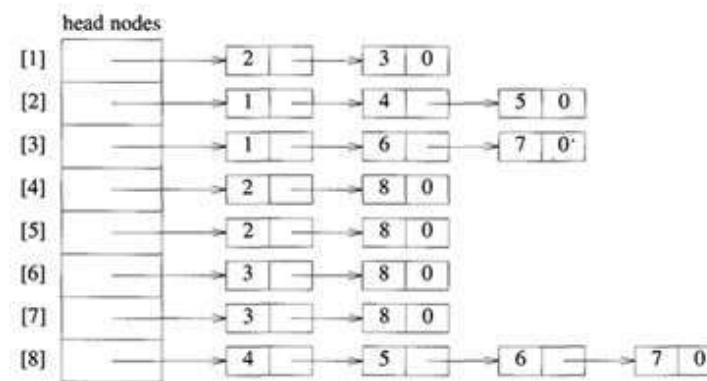
In DFS nodes are explored in the order  
= ?

# DISCUSSION

- Why we use queue in BFS and stack in DFS ?
- What are the differences between BFS and DFS ?
- What are the time complexity of BFS and DFS ?
- Why time complexity differs while using adjacency list rather than using adjacency matrix ?
- Who explores/ visits the leaf node first, DFS or BFS ?
- Who explores/ visits all of the same level nodes before going to next level, DFS or BFS ?
- What will be the traversing order using DFS, started from vertex 4, for the following graph and adjacency list?



(a) Undirected graph  $G$



(c) Adjacency list for  $G$